Agentic AI Capstone Project Documentation

E-Commerce Customer Support Assistant

Name: Aastha

Roll Number: _____

Batch/Program: _____

Problem Statement

E-commerce support teams handle a high volume of repetitive customer queries about returns, refunds, shipping timelines, order tracking, and damaged items. Manual handling increases response time and introduces inconsistency in policy communication. This project builds an agentic assistant that retrieves grounded policy answers, uses tools for operational queries, and preserves context across multi-turn conversations.

Solution Overview

The solution is implemented as a LangGraph-based stateful workflow with strict phase-wise design. A ChromaDB in-memory knowledge base stores ten domain documents, embedded with all-MiniLM-L6-v2. The graph routes each query through retrieve, tool, or skip paths, generates grounded answers, applies an evaluation gate for faithfulness, and writes responses back to conversation memory using thread_id persistence.

Core Features

1) RAG policy assistant with top-3 retrieval chunks and source topics. 2) Memory-aware conversation continuity using MemorySaver and thread IDs. 3) Tool node support for datetime and calculator requests. 4) Faithfulness scoring with retry loop and retry limits. 5) Safe handling for out-of-scope and prompt injection requests.

Architecture and Workflow

Graph flow: memory -> router -> retrieve/tool/skip -> answer -> eval -> save -> END. The memory node appends user messages with sliding window control and entity extraction. The router chooses the appropriate route. Retrieval adds contextual policy chunks; tool provides external computation/time signals. The answer node enforces strict grounding. The eval node computes faithfulness and conditionally retries before final save.

Tech Stack

Python, LangGraph, LangChain, ChromaDB, SentenceTransformers (all-MiniLM-L6-v2), Streamlit, Groq LLM integration, and fallback local runtime for resilience.

Testing and Validation

The project includes retrieval gate checks before graph execution, isolated node testing, phase-5 scenario tests with PASS/FAIL judgment, dedicated memory sequence testing, red-team prompts (out-of-scope and prompt-injection), and phase-6 evaluation with RAGAS-compatible fallback scoring when RAGAS is unavailable.

Unique Points

The system is structured to match capstone warning constraints: retrieval-first gating, typed state safety, non-crashing tool behavior, explicit END path in graph, and UI continuity via session state. It also includes automatic LLM fallback if API key issues occur, ensuring the app remains operable.

Screenshots

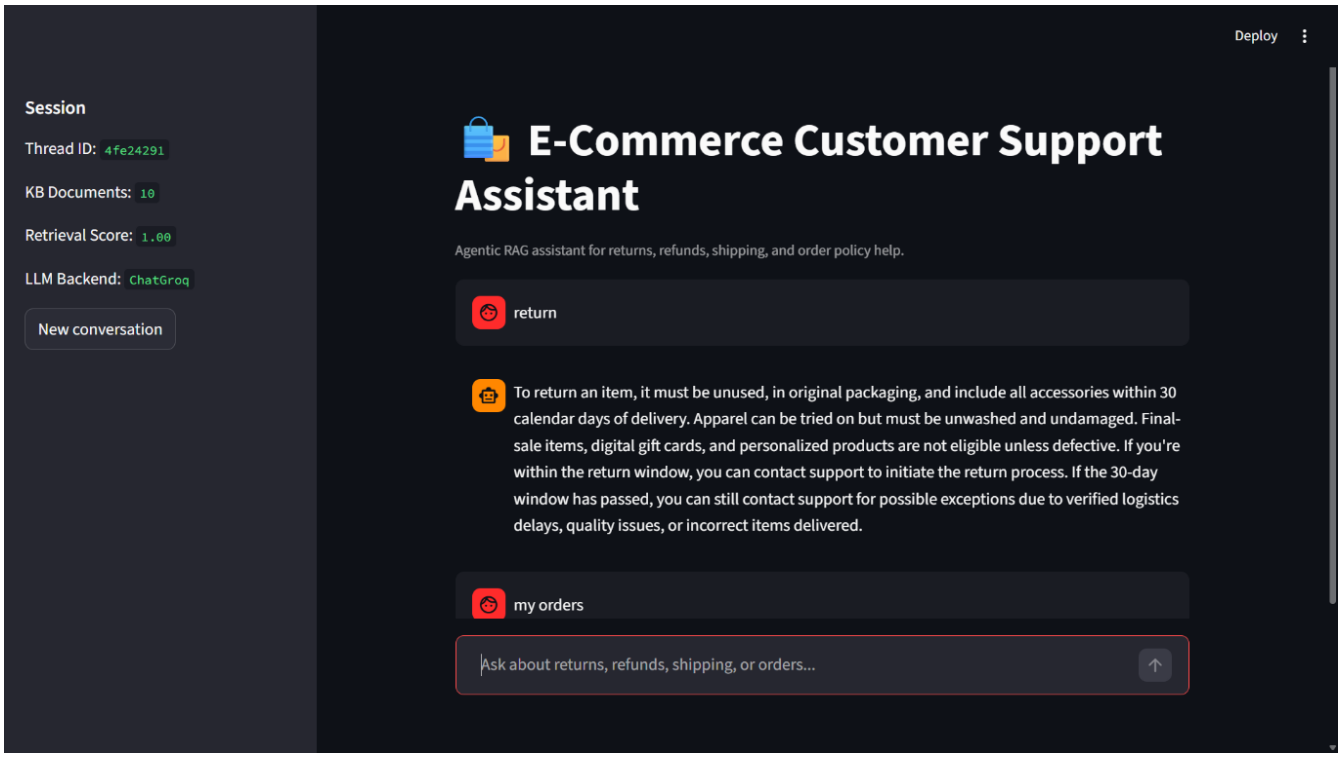

Figure: Streamlit interface showing active session metrics, retrieval score, backend, and chat responses.

Future Improvements

1) Hybrid retrieval (BM25 + dense vectors) with reranking for better precision. 2) Structured citations with sentence-level evidence mapping. 3) Real order-system API integration for live order lookup. 4) Expanded safety and compliance policy checks with confidence thresholds. 5) Continuous evaluation dashboard tracking score trends over versions.

Submission Artifacts

Code deliverables include agent.py, capstone_streamlit.py, completed notebook, and README. This document is the project PDF artifact required by the submission guidelines.